

Lab05_Spring_Been_Scope

1. GET All Coffees

ดึงข้อมูลเมนูกาแฟทั้งหมดที่มีอยู่ในระบบ

GET

http://localhost:8080/coffees

Send

Docs

Params

Authorization

Headers 7

Body

Scripts

Settings

Cookies

Query Params

Key	Value	Description	Bulk Edit	...
Key	Value	Description		

Body

Cookies

Headers 5

Test Results

200 OK

16 ms

234 B

Save Response

...

{ } JSON

Preview

Visualize

```
1  [
2    {
3      "id": 1,
4      "name": "Espresso",
5      "price": 45.0
6    },
7    {
8      "id": 2,
9      "name": "Latte",
10     "price": 55.0
11   }
12 ]
```

2. GET Coffee by ID

ค้นหาข้อมูลเมนูกาแฟตามรหัส (ID) ที่ระบุ

The screenshot shows a REST client interface with the following components:

- Request Bar:** Method: GET, URL: `http://localhost:8080/coffees/1`, Send button.
- Params Tab:** Shows a table for Query Params.
- Response Bar:** Status: 200 OK, Time: 5 ms, Size: 203 B, Save Response button.
- Body Tab:** Shows the JSON response.

Query Params Table:

Key	Value	Description	Bulk Edit	...
Key	Value	Description		

JSON Response:

```
1  {
2    "id": 1,
3    "name": "Espresso",
4    "price": 45.0
5  }
```

3. POST Create Coffee

เพิ่มเมนูกาแฟรายการใหม่เข้าสู่ระบบ โดยส่งข้อมูลในรูปแบบ JSON

The screenshot shows a REST client interface with the following details:

- Method:** POST
- URL:** http://localhost:8080/caffeas
- Body Type:** raw
- Request Body (JSON):**

```
1 {  
2   "name": "Cappuccino",  
3   "price": 60.0  
4 }
```
- Response Status:** 200 OK
- Response Time:** 139 ms
- Response Size:** 205 B
- Response Body (JSON):**

```
1 {  
2   "id": 3,  
3   "name": "Cappuccino",  
4   "price": 60.0  
5 }
```

มีเมนู Cappuccino เพิ่มเข้ามาใหม่

GET

http://localhost:8080/coffees

Send

Docs

Params

Authorization

Headers 9

Body

Scripts

Settings

Cookies

Query Params

Key	Value	Description	Bulk Edit	...
Key	Value	Description		

Body

Cookies

Headers 5

Test Results

200 OK

8 ms

277 B

Save Response

{ } JSON

Preview

Visualize

```
1  [
2    {
3      "id": 1,
4      "name": "Espresso",
5      "price": 45.0
6    },
7    {
8      "id": 2,
9      "name": "Latte",
10     "price": 55.0
11   },
12   {
13     "id": 3,
14     "name": "Cappuccino",
15     "price": 60.0
16   }
17 ]
```

4. PUT Update

แก้ไขข้อมูลเมนูกาแฟที่มีอยู่ โดยระบุ ID และข้อมูลใหม่

The screenshot shows a REST client interface with a PUT request to `http://localhost:8080/coffees/2`. The request body is a JSON object: `{ "id": 2, "name": "Latte", "price": 50.0 }`. The response is a `200 OK` status with a response time of 19 ms and a body size of 200 B. The response body is also shown as a JSON object: `{ "id": 2, "name": "Latte", "price": 50.0 }`.

PUT `http://localhost:8080/coffees/2` Send

Docs Params Authorization Headers 9 Body Scripts Settings Cookies

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL JSON Schema Beautify

```
1 {
2   "id": 2,
3   "name": "Latte",
4   "price": 50.0
5 }
```

Body Cookies Headers 5 Test Results 200 OK • 19 ms • 200 B | Save Response

{ } JSON Preview Visualize

```
1 {
2   "id": 2,
3   "name": "Latte",
4   "price": 50.0
5 }
```

เมนู Latte ได้รับการแก้ไขราคาใหม่จาก 55.0 เป็น 50.0

GET

http://localhost:8080/coffees

Send

Docs

Params

Authorization

Headers 9

Body

Scripts

Settings

Cookies

Query Params

Key	Value	Description	Bulk Edit	...
Key	Value	Description		

Body

Cookies

Headers 5

Test Results

200 OK

3 ms

277 B

Save Response

JSON

Preview

Visualize

```
1  [
2    {
3      "id": 1,
4      "name": "Espresso",
5      "price": 45.0
6    },
7    {
8      "id": 2,
9      "name": "Latte",
10     "price": 50.0
11   },
12   {
13     "id": 3,
14     "name": "Cappuccino",
15     "price": 60.0
16   }
17 ]
```

5. DELETE

ลบข้อมูลเมนูกาแฟตามรหัส (ID) ที่กำหนดออกจากระบบ

DELETE http://localhost:8080/coffees/3

Send

Docs Params Authorization Headers 9 Body • Scripts Settings Cookies

Query Params

Key	Value	Description	Bulk Edit	...
Key	Value	Description		

Body Cookies Headers 4 Test Results 200 OK • 4 ms • 123 B • Save Response ...

Raw Preview Visualize 1

ตรวจสอบรายการเมนูกาแฟทั้งหมดอีกครั้ง เพื่อยืนยันว่าข้อมูลถูกเพิ่ม แก้ไข และลบเรียบร้อยแล้ว

GET http://localhost:8080/coffees

Send

Docs Params Authorization Headers 9 Body • Scripts Settings Cookies

Query Params

Key	Value	Description	Bulk Edit	...
Key	Value	Description		

Body Cookies Headers 5 Test Results 200 OK • 4 ms • 234 B • Save Response ...

{ } JSON Preview Visualize

```
1 [
2   {
3     "id": 1,
4     "name": "Espresso",
5     "price": 45.0
6   },
7   {
8     "id": 2,
9     "name": "Latte",
10    "price": 50.0
11  }
12 ]
```

Discussion

1. HTTP Method แต่ละตัว (GET/POST/PUT/DELETE) ต่างกันอย่างไร ยกตัวอย่างจากโปรเจกต์ตัวเอง

ตอบ HTTP Method แต่ละตัวมีหน้าที่แตกต่างกันตามการจัดการข้อมูลในระบบ โดย GET ใช้สำหรับดึงข้อมูล เช่น GET /coffees เพื่อแสดงเมนูกาแฟทั้งหมด และ GET /coffees/{id} เพื่อค้นหากาแฟตามรหัส POST ใช้เพิ่มข้อมูลใหม่ เช่น POST /coffees เพื่อเพิ่มเมนูกาแฟ PUT ใช้แก้ไขข้อมูลเดิม เช่น PUT /coffees/{id} เพื่อแก้ไขชื่อหรือราคา และ DELETE ใช้ลบข้อมูล เช่น DELETE /coffees/{id} เพื่อลบเมนูกาแฟออกจากระบบ

2. ทำไมต้องแยก Controller กับ Service ออกจากกัน มีข้อดีอย่างไรถ้าโปรแกรมโตขึ้น

ตอบ การแยก Controller และ Service ทำให้แต่ละส่วนมีหน้าที่ชัดเจน โดย Controller ทำหน้าที่รับคำขอจากผู้ใช้และส่งผลลัพธ์กลับ ส่วน Service เป็นส่วนที่จัดการตรรกะการทำงานของระบบ เช่น การค้นหา เพิ่ม แก้ไข และลบข้อมูล เมื่อโปรแกรมมีขนาดใหญ่จะช่วยให้โค้ดเป็นระเบียบ แก้ไขหรือพัฒนาเพิ่มเติมได้ง่าย และสามารถนำ Service ไปใช้ซ้ำกับ Controller อื่นได้

3. ข้อมูลที่เก็บไว้ใน List ใน Memory หายไปตอนไหน และถ้าอยากให้ไม่หายควรทำอย่างไร

ตอบ ข้อมูลที่เก็บไว้ใน List จะหายไปเมื่อปิดโปรแกรมหรือหยุดการทำงานของ Spring Boot เพราะข้อมูลถูกเก็บไว้ในหน่วยความจำ (Memory) เท่านั้น หากต้องการให้ข้อมูลไม่หาย ควรเก็บข้อมูลลงฐานข้อมูล เช่น MySQL หรือ PostgreSQL เพื่อให้ข้อมูลยังคงอยู่แม้จะปิดและเปิดโปรแกรมใหม่

4. @RestController, @GetMapping, @PostMapping, @PathVariable, @RequestBody แต่ละตัวทำหน้าที่อะไร

- @RestController ใช้กำหนดให้คลาสเป็น Controller สำหรับให้บริการ REST API และส่งข้อมูลกลับในรูปแบบ JSON
- @GetMapping ใช้กำหนดเมธอดที่ทำงานเมื่อมีการเรียก HTTP GET เพื่อดึงข้อมูล
- @PostMapping ใช้กำหนดเมธอดที่ทำงานเมื่อมีการเรียก HTTP POST เพื่อเพิ่มข้อมูลใหม่
- @PathVariable ใช้รับค่าที่ส่งมาทาง URL เช่น id เพื่อนำไปใช้งานภายในเมธอด
- @RequestBody ใช้รับข้อมูลที่ส่งมาในรูปแบบ JSON จาก Request แล้วแปลงเป็น Object เพื่อนำไปประมวลผล